Interfaces

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **write** a class that implements an interface.
2. Be able to **use** an interface as a type, like `ArrayList<Playable>`.
3. Be able to **implement** `Comparable` to sort an `ArrayList`.

New week, new app

```
public abstract class Track {          // like Book
  private String title;
  private String artist;
  private int seconds;

  public void play() { ... }           // "Now playing: ..."
}

public class Song extends Track { ... }      // a kind of Track
public class Podcast extends Track { ... }   // another kind
```

Same moves as last week's library: an abstract class, and kinds of it.

...and a `Playlist`

```
public class Playlist {
  private String name;
  private Track[] tracks;   // an oversize array...
  private int size;         // ...and its size
}
```

Should `Playlist` extend `Track`? **No!** A playlist *has* tracks.

One Play button

```
public static void press(Track t) {
  t.play();
}

Song jumpAround = new Song("Jump Around", ...);
Playlist roadTrip = new Playlist("Road Trip", 10);
press(jumpAround);   // fine: a Song is a Track
press(roadTrip);     // not applicable for the arguments (Playlist)
```

You can press play on a `Track` *or* a `Playlist`. But Java has no type for *"anything that plays"*... **yet.**

Interfaces

Definition: An **interface** is a list of methods that a class promises to have.

```
public interface Playable {
  public void play();   // no body! Just the promise.
}
```

No fields, no constructors, no method bodies. It goes in its own file, `Playable.java`.

`implements`

```
public class Playlist implements Playable {
```

```
The type Playlist must implement the inherited abstract method Playable.play()
```

Red already? **We promised a `play()`, and haven't written one!**

`implements`

```
public class Playlist implements Playable {
  @Override
  public void play() {   // keep the promise
    ...                  // play every track, in order
  }
}
```

It must be `public`. Leave that off and you get: *Cannot reduce the visibility of the inherited method from Playable*.

Eclipse Exercise

Get today's starter from Canvas.

TODO 1a. Write the `Playable` interface.

TODO 1b. Make `Track` and `Playlist` Playable.

One Play button, for real

```
public static void press(Playable p) {
  p.play();
}

Song jumpAround = new Song("Jump Around", ...);
Playlist roadTrip = new Playlist("Road Trip", 10);
press(jumpAround);   // a Song: plays one track
press(roadTrip);     // a Playlist: plays every track
```

An interface is a **type**. A `Playable` variable holds anything that keeps the promise.

...but no `new Playable()`

```
ArrayList<Playable> upNext = new ArrayList<Playable>();   // fine
Playable p = new Playable();   // Cannot instantiate the type Playable
```

An interface has no code to run. Like an abstract class, `new` can't make one.

Eclipse Exercise

TODO 2. Fill Up Next with a song, the road trip, and a podcast. Then play everything in it.

Abstract class or interface?

	`Track`: **abstract class**	`Playable`: **interface**
Fields? Constructors?	Yes	No
How many per class?	One: `extends`	Many: `implements`

Tophat Q1

A quick check on `extends` and `implements`.

→ Open Tophat to vote.

Let's sort the library

```
Collections.sort(library);   // library is an ArrayList<Song>
```

```
The method sort(List<T>) in the type Collections is not applicable
for the arguments (ArrayList<Song>)
```

By title? By length? By plays? Java can't know. **You decide.**

`Comparable`

Java's own interface, with one method:

```
public interface Comparable<T> {
  public int compareTo(T other);
}
```

What's that `<T>`? **The type to compare against.**

More on Wednesday!

`compareTo` answers with a number

```
a.compareTo(b) < 0      // a goes first
a.compareTo(b) == 0     // a tie
a.compareTo(b) > 0      // b goes first

"Bohemian Rhapsody".compareTo("Jump Around")   // negative: B before J
```

Not a `boolean`. Only the sign matters.

Song implements Comparable<Song>

```
public class Song extends Track implements Comparable<Song> {
  @Override
  public int compareTo(Song other) {
    return this.getTitle().compareTo(other.getTitle());   // A to Z
  }
}
```

We decided: titles. Now `Collections.sort(library)` works!

Eclipse Exercise

TODO 3a. Songs sort A to Z by title.

TODO 3b. Podcasts sort by episode number -- on your own!

TODO 3c. Sort both lists with `Collections.sort`.

Tophat Q2

Song's `compareTo` is now:

```
@Override
public int compareTo(Song other) {
  return other.getPlays() - this.getPlays();
}
```

After `Collections.sort(library)`, which song is first?

→ Open Tophat to vote.

Questions?